

Boris Cherny 与 Claude Code：诞生场景、设计思想和产品演进机制

2026-08-25 • 16 Sources

16

交叉核验来源

24

证据条目

6

核心判断

研究日期：2026-08-25

研究模式：Standard

结论置信度：高（人物与产品历史主要来自 Boris 本人、Anthropic 官方口述史、官方文档；组织内部优先级细节只能依据公开访谈）

Executive Summary

Boris Cherny 是 Anthropic 的工程师、Claude Code 的创造者和现任负责人；此前长期在 Meta/Instagram 从事产品、基础设施与代码质量工作，也是 O'Reilly《Programming TypeScript》的作者。[1][15][16] 但“Boris 一个人从零发明了全部 Claude Code”并不准确：Anthropic 从 2021—2023 年已在做 coding RL、执行环境、工具调度和内部 clide；Boris 在 2024 年把这些已经成熟的模型能力与一个新的、从头实现的 Claude CLI 产品形态组合起来，形成了现代 Claude Code 的直接起点。[2]

Claude Code 的核心洞察不是“把聊天框搬进终端”，而是发现模型已经能自主探索文件系统、选择 Bash/文件工具、根据结果继续行动；产品的任务是把这种能力释放出来，而不是用厚重的流程把模型固定在预设路径里。[2][3][4] 因而它采用薄 harness：模型负责推理与动态编排，工具负责行动，Claude Code 负责上下文、执行环境、权限与反馈循环。[8][11][14]

Skills、MCP、Hooks、Subagents 等是这一思想的后续分层，而不是 Boris 在首个两天原型里一次性设计完成的功能。它们分别解决按需知识与流程、外部连接、确定性生命周期控制、上下文隔离与并行执行；其共同原则是简单、可组合、可渐进加载，并让模型在开放工具空间中决定下一步。[9][10]

产品演进不是传统的季度路线图驱动，而是五类信号共同作用：内部 dogfood、直接用户反馈、行为与遥测、用户“误用”所暴露的潜在需求、未来几个月的模型能力曲线。团队对修复和粗糙边缘极快响应，但对净新增功能保持高门槛：必须直觉化、低学习成本，并与“更强模型将获得更大自主性”的方向一致。[3][5] 2026 年的质量事故又补上了更严格的公共版本 dogfood、逐模型评测、消融、浸泡期和渐进发布机制。[13]

分析关键词：thin harness, agent loop, latent demand, dogfood, progressive disclosure, tool composition, evaluation, safety boundary.

Introduction

Research Question

本报告回答五个问题：Boris Cherny 是谁；Claude Code 在什么场景下诞生；它最初抓住了什么模型能力；Skills 与各种工具怎样组成一个系统；团队依据什么原则更新产品、需求从哪里来。

Scope & Methodology

研究优先使用 2024—2026 年的一手或准一手资料：Anthropic 2026 年官方口述史、Boris 与 Cat Wu 的 Latent Space 访谈、Boris 的 Lenny's Podcast 访谈、Anthropic 产品公告与当前官方文档。Pragmatic Engineer 的深访用于补足内部 dogfood 数字与工程过程；个人主页、O'Reilly 和 GitHub 用于人物信息交叉验证。[1][2][3][4][5][15][16]

“设计思想”分为三层：Boris 明确说过的原则；Anthropic 团队公开的产品与 agent 工程原则；从当前架构中可验证的实现映射。第三类均标为综合判断，不把后来由更大团队完成的功能归于 Boris 一人。

Key Assumptions

- “创造 Claude Code”指现代 Claude Code 产品与其直接原型，不否认 Anthropic 更早的 coding RL、执行环境和 clide。
- 公开访谈能说明方向和方法，但不能完整还原内部优先级、商业约束和未公开评测。
- Skills、Hooks、Agent Teams 等以 2026-08-25 的公开产品形态为准；它们晚于首个 Claude CLI 原型。

Main Analysis

Finding 1: Boris 是“现代 Claude Code 的产品创造者”，不是所有底层技术的唯一发明者

Boris 的自述很克制：他是 Anthropic 的软件工程师，在那里创造了 Claude Code；此前在 Instagram 做产品与基础设施原型和规模化工作；2019 年出版《Programming TypeScript》，目前居住在旧金山。[1][15] 他的 GitHub 长期聚焦 TypeScript、类型系统、前端基础设施和开源工具，这与后来选择 TypeScript/React 构建 Claude Code、偏爱简单开发者工具的风格具有一致性，但这种一致性只能作为背景，不当作因果证明。[16]

Anthropic 2026 年官方口述史修正了一个常见的英雄叙事。早在 2021—2022 年，Anthropic 研究团队就在训练能写函数、执行测试和做更开放软件工程任务的模型，也处理过持久 shell、流式 I/O、超时与安全执行环境。2023 年前后，公司内部已有 clide，能聊天式编辑代码和做开发任务；它很超前，但启动慢、使用繁琐，且不同阶段的工具能力并不完整。[2]

Boris 在 2024 年 9 月加入 Anthropic Labs 后，被分配的方向是“自动化编码”。他先通过公共 API 学习模型，另起炉灶做了 Claude CLI。最初 Slack 帖子已经列出今天仍可辨认的骨架：REPL、Unix 管道、读写文件、Bash、屏幕与网络工具、插件、上下文和按实例/目录授权的权限模型。原型约两天完成，演示的是让 Claude 判断当前播放的音乐；最初只得到少量 Slack 反应。[2]

因此更准确的表述是：Anthropic 已经积累了模型训练、agent harness 和内部工具的技术土壤；Boris 识别并组合了决定现代 Claude Code 形态的产品原语，并以极快反馈把它推成独立产品。这个区别重要，因为 Claude Code 的成功既来自个人产品判断，也来自模型、研究和内部工具多年积累。

Finding 2: 诞生不是路线图执行，而是“模型潜在能力 + 内部真实使用”被连续发现

首个原型没有宏大的产品规划。Boris 在访谈中明确表示，当时是在试验 Claude 能出现在哪些地方；音乐和屏幕理解只是探索。转折发生在他给模型终端和编码能力之后：模型会自主读取一个文件、沿 import 继续找相关文件、运行命令并根据结果迭代。Boris 把这种“模型已经会，但尚无产品释放它”的差距称为 product overhang。[3][4]

验证不是问卷，而是 dogfood。Boris 先每天使用，核心团队随后每天使用，再扩到 Anthropic 的工程师和研究员。公开深访给出的数字是：2024 年 11 月内部可用版本上线后，首日约 20% 工程团队使用，第五天约 50%；Latent Space 访谈则描述内部 DAU 连续陡增，成为向外发布的直接信号。[3][4]

产品于 2025 年 2 月 24 日以 limited research preview 发布。官方目标不仅是提高开发效率，也包括理解开发者怎样使用 Claude、把观察反哺模型能力与安全。[6] 2025 年 5 月 22 日，在预览反馈后转为 GA，并扩展到 IDE 和后台任务。[7] 这解释了为何 Claude Code 一开始保持低层、粗粝：它首先是一个“观察模型与用户共同发明工作流”的研究产品，其次才逐渐成为成熟开发工具。

Finding 3: 核心设计哲学是薄 harness、强模型、开放工具空间

当前官方文档把 Claude Code 描述为循环：收集上下文、行动、验证，再根据每次工具结果继续调整。模型负责推理，工具负责行动，harness 提供工具、上下文管理和执行环境。[8] 这不是后来才添加的解释；Boris 的首个原型已经把文件、Bash、屏幕、网络、上下文和权限并列为基础原语。[2]

薄 harness 有三层含义。第一，尽量不替模型写死 workflow。Anthropic 对 workflow 与 agent 的区分是：workflow 由代码预设路径，agent 由模型动态决定过程和工具使用；其经验是从最简单可行方案开始，仅在必要时增加复杂度。[11] 第二，产品尽量提供原语而非封闭体验。终端天然继承现有命令、脚本、Git、管道和团队环境；Claude Code 因此更像 Unix utility，而不是要求用户迁移到一个新世界。[3][14] 第三，面向未来模型设计。团队讨论路线图时，不只问今天用户最容易接受什么，还问几个月后的模型会更擅长探索、完成复杂任务和组合工具时，当前产品是否仍能放大它。[3]

“简单”不等于没有工程。权限是早期最复杂的部分之一；随着自主性提高，产品又引入文件系统和网络隔离。Anthropic 报告沙箱在内部把权限提示减少了 84%，体现了一个重要平衡：在可验证边界内放大自主性，而不是用频繁确认把自主性压回去。[12] 这也说明安全不是外加限制，而是能力释放的前提。

Finding 4: Skills 与工具组合，是对 agent loop 的职责分层

Skills 不是 Claude Code 诞生时的单一“秘密武器”，而是 2025 年后形成的扩展层。Anthropic 把 Skill 定义为包含 SKILL.md、可选脚本和资源的目录：启动时只给模型名称与描述，相关时再加载正文，必要时继续读取引用文件或执行脚本。这种 progressive disclosure 让知识规模不必全部挤进上下文，也把确定性计算交给代码。[10]

从职责看，各能力不是互相替代，而是组合：CLAUDE.md 提供每次会话都需要的项目约定；Skill 提供按需知识、检查清单和可复用流程；MCP 提供数据库、SaaS 或浏览器等外部连接；Hook 在编辑、工具调用、压缩或会话结束等生命周期点执行确定性规则；Subagent 把大量探索放入独立上下文，只把摘要带回；Agent Team 则让多个独立会话协同。[9]

这套分层背后有一个统一判断：让概率模型负责需要理解、规划和权衡的部分；让程序、权限、Hook 和沙箱负责必须稳定发生或绝不能发生的部分。Skill 教模型“怎样做”；MCP 给模型“能调用什么”；Hook 与权限规定“什么必须/不得发生”；Subagent 解决“在哪个上下文里做”。官方文档也明确提醒，提示词中的禁止条款不是保证，真正的强约束应该进入权限或 Hook。[9]

因此，所谓“各种工具的组合”不是把工具越接越多，而是控制上下文成本并保持职责清晰：基础文件/搜索/Bash 足以覆盖大部分编码；Skill 在需要时加载方法；MCP 只负责外部能力；复杂探索交给隔离上下文。模型在每次工具结果后选择下一步，组合性来自循环，而不是预先绘制的巨型流程图。[8][9][10]

Finding 5: 更新产品的原则，是追随能力曲线、放大潜在需求、快速闭环，同时严控新增复杂度

公开材料显示，Claude Code 的路线图很少是纯 top-down。Cat Wu 形容 PM 介入较轻，许多功能来自长期使用产品的团队成员去做自己希望存在的能力；PM 更多清除法律、营销与跨团队障碍。长期方向由全队共同判断未来模型会更擅长什么，并保证产品形态与那个未来兼容。[3]

需求优先级可以归纳为六个来源：

1. **内部 dogfood**：工程师、研究员、设计师、数据科学家真实使用，最早的 PM 也因持续提交大段反馈加入团队。[2][3]
2. **直接用户反馈**：/feedback、GitHub 集中讨论、X 上的 bug 与 feature request，以及访谈和 shadowing。Boris 早期会在几分钟内修复反馈，这种“用户立刻感到被听见”的循环反过来增加反馈量。[5][13]
3. **客服、销售、合规和企业需求**：web fetch 等功能既是高频请求，也必须与法律、安全和企业部署要求共同设计；支持和成功团队的频道会直接暴露长期 backlog 中的粗糙边缘。[3]
4. **行为数据与遥测**：团队关注从首次 commit 到 PR 合并的 cycle time、原本不会被实现的功能数，并让 Claude 开始阅读反馈、bug report 和 telemetry 来提出修复建议。[3][5]
5. **潜在需求**：观察用户如何绕过原设计使用产品。例如非工程人员用终端做 SQL、基因组、照片恢复或植物监测，后来推动了更通用的 Cowork；同样也观察“模型想做什么”，给它更合适的工具而不是把它塞进固定步骤。[5]
6. **模型与安全研究**：公开发布让团队看到实验室评测覆盖不到的真实行为；产品改动也要服务于更强模型，而不是永久补丁化今天的能力缺口。[5][6]

但高速度不等于无边界加功能。团队对 bug、边缘情况和摩擦迅速修复，对净新增功能却要求高：交互必须直觉、上手成本低、符合产品愿景。与其长时间写设计文档，团队常让 Claude Code 生成多个可运行原型，用实际体验决定抽象与交互。[3] 这是一种“把实现成本下降转化为更高验证密度”的方法，而不是把下降的成本全部转化为更多永久功能。

2026 年质量事故进一步暴露了快速迭代的另一面。默认 reasoning effort、思考历史清理和系统提示三类改动产生了不同退化，内部使用和原评测未能及时复现。团队随后承诺让更多员工使用与公众完全一致的 build，并对每个模型的系统提示改动运行更广评测、消融、浸泡期和渐进发布。[13] 因而成熟后的更新原则应概括为：发现快、原型快、修复快；但默认值、上下文和系统提示等高杠杆改动必须慢发布、强评测。

Synthesis & Insights

Pattern 1: Claude Code 的真正产品不是 CLI，而是“模型能力的低损耗传输层”

终端只是最初最合适的载体。核心竞争力在于减少从模型能力到真实行动之间的损耗：给出文件、搜索、执行和验证工具，让模型控制顺序；用 CLAUDE.md/Skills 提供刚好够用的上下文；用权限、Hook 和沙箱控制风险。只要这个 agent loop 保持一致，它可以出现在终端、IDE、桌面、Web 或移动端。[8][9]

Pattern 2: 需求有两类主体——用户和模型

传统产品只观察用户想做什么；Claude Code 还观察模型已经试图做什么。用户的异常用法暴露 latent demand，模型的自主探索暴露 product overhang。产品更新是在二者交叉点上提供一个更顺滑、更安全的表面。这是 Boris 产品观里最值得复用的部分：不要只问“用户要哪个按钮”，也要问“用户已经在绕过什么限制，模型已经在绕过什么脚手架”。[4][5]

Pattern 3: 实现变便宜后，稀缺资源从编码转向判断、评测和风险边界

Claude Code 团队能在很短时间内做出多个原型，需求并未因此自动变得正确。新的瓶颈是：选择哪个问题、定义何为完成、验证跨模型与长会话质量、控制安全边界、避免产品复杂度污染模型能力。2026 年 postmortem 说明，快速生成代码并不能替代公开构建一致性、回归评测和渐进发布。[13]

Claims–Evidence Table

核心判断	证据类型	置信度
Boris 创造了现代 Claude Code 产品	本人主页、官方口述史	高
Claude Code 建立在更早 coding RL 与 clide 经验上	Anthropic 官方口述史	高
现代直接原型始于 2024 年两天左右的 Claude CLI	官方口述史、两份访谈	高
薄 harness 与最小脚手架是核心设计方向	Boris/Cat 访谈、官方文档	高
Skills 是后续扩展层，不是首个原型的完整形态	官方时间线与 Skills 发布材料	高
路线图以 bottom–up 使用和未来模型能力共同驱动	Boris/Cat 访谈	中高
潜在需求是重要产品信号	Boris 2026 访谈和案例	中高
高杠杆更新已转向更强 eval 与渐进发布	2026 官方 postmortem	高

Limitations & Caveats

第一，Boris 和 Anthropic 的访谈兼具复盘与产品传播属性；“coding is solved”等表述是个人立场，而且 Boris 自己也限定为他所做的编程类型，不能推广为行业事实。[5] 本报告没有用该表述作为核心结论。

第二，Skills、MCP、Hooks、Subagents 的公开文档能证明当前职责分层，但不能证明每一项都由 Boris 亲自提出或拍板。报告只把它们视为与早期哲学连续的团队化产品演进。[9][10]

第三，内部采纳率、发布频率等细节主要来自访谈型二手报道，虽直接采访创始团队，仍不等同于经审计运营数据。[4] 本报告仅用它们说明增长方向，不据此做财务或市场判断。

Counterevidence Register

对“Claude Code 是 Boris 两天内从零发明”的反证，是 Anthropic 早在 2021—2023 年已有 coding RL、执行环境和 clide。[2] 对“越快发布越好”的反证，是 2026 年三类质量退化穿过了既有测试与 dogfood，迫使团队加强公共构建一致性、评测与灰度。[13] 对“模型自主性应无限扩大”的反证，是权限疲劳、提示注入和本地执行风险推动了沙箱与网络/文件边界。[12] 这些反证没有推翻主结论，而是把它限定为：薄 harness 必须建立在强验证和硬边界之上。

Recommendations

如果把 Boris/Claude Code 的方法迁移到自己的 Agent 产品，建议依次做四件事：先建立最薄的“模型—工具—结果—再决策”循环；用真实内部任务 dogfood，而不是先堆功能；把重复方法放入 Skill、外部能力放入 MCP、强制规则放入 Hook/权限、嘈杂探索放入 Subagent；最后建立“快原型、慢默认值”的发布机制，对 prompt、context、权限和模型路由做逐模型 eval、浸泡和灰度。

更重要的是建立双重需求观测：一条线记录用户主动提出的问题；另一条线记录用户绕路完成的任务、模型反复尝试的动作和工具缺口。前者告诉你哪里痛，后者更可能告诉你下一代产品是什么。

Methodology Appendix

本次研究于 2026-08-25 完成。检索分为人物履历、产品起源、架构/扩展、产品更新与反馈机制四组，共登记 16 个来源、24 条证据。核心历史优先采用 Anthropic 官方口述史和 Boris/Cat 的原始访谈；官方文档用于验证当前架构；二手深访只补足官方材料没有量化的内部过程。

交叉验证时，特别处理了两个容易混淆的问题：一是 clide 与 Boris 的 Claude CLI 不是同一阶段、同一产品；二是首个原型的插件/工具概念与后来标准化的 Agent Skills 不应混为一谈。报告大纲因此从单一“Boris 发明史”调整为“前置技术积累—现代产品原型—团队化扩展—更新机制”。

BIBLIOGRAPHY

[1] Boris Cherny (2026). "Boris Cherny's Blog: About". Personal website. <https://borischerny.com/about/> (Retrieved: 2026-08-25)

[2] Anthropic (2026). "The Making of Claude Code". Anthropic Features. <https://www.anthropic.com/features/making-of-claude-code> (Retrieved: 2026-08-25)

[3] Latent Space (2025). "Claude Code: Anthropic's Agent in Your Terminal". Interview with Boris Cherny and Cat Wu. <https://www.latent.space/p/claude-code> (Retrieved: 2026-08-25)

[4] Gergely Orosz (2025). "How Claude Code is built". The Pragmatic Engineer. <https://newsletter.pragmaticengineer.com/p/how-claude-code-is-built> (Retrieved: 2026-08-25)

[5] Lenny Rachitsky (2026). "Head of Claude Code: What happens after coding is solved". Lenny's Podcast. <https://www.lennysnewsletter.com/p/head-of-claude-code-what-happens> (Retrieved: 2026-08-25)

[6] Anthropic (2025). "Claude 3.7 Sonnet and Claude Code". Anthropic News. <https://www.anthropic.com/news/claude-3-7-sonnet> (Retrieved: 2026-08-25)

[7] Anthropic (2025). "Introducing Claude 4". Anthropic News. <https://www.anthropic.com/news/claude-4> (Retrieved: 2026-08-25)

[8] Anthropic (2026). "How Claude Code works". Claude Code Documentation. <https://code.claude.com/docs/en/how-claude-code-works> (Retrieved: 2026-08-25)

[9] Anthropic (2026). "Extend Claude Code". Claude Code Documentation. <https://code.claude.com/docs/en/features-overview> (Retrieved: 2026-08-25)

[10] Anthropic (2025). "Equipping agents for the real world with Agent Skills". Anthropic Engineering. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills> (Retrieved: 2026-08-25)

- [11] Anthropic (2024). "Building effective agents". Anthropic Engineering. <https://www.anthropic.com/engineering/building-effective-agents> (Retrieved: 2026-08-25)
- [12] Anthropic (2025). "Beyond permission prompts: making Claude Code more secure and autonomous". Anthropic Engineering. <https://www.anthropic.com/engineering/claude-code-sandboxing> (Retrieved: 2026-08-25)
- [13] Anthropic (2026). "An update on recent Claude Code quality reports". Anthropic Engineering. <https://www.anthropic.com/engineering/april-23-postmortem> (Retrieved: 2026-08-25)
- [14] Anthropic (2025). "Claude Code: Best practices for agentic coding". Anthropic Engineering. <https://www.anthropic.com/engineering/claude-code-best-practices> (Retrieved: 2026-08-25)
- [15] Boris Cherny (2019). "Programming TypeScript". O'Reilly Media. <https://www.oreilly.com/library/view/programming-typescript/9781492037644/> (Retrieved: 2026-08-25)
- [16] Boris Cherny (2026). "Boris Cherny GitHub profile". GitHub. <https://github.com/bcherny> (Retrieved: 2026-08-25)